

ITA0613 - Machine Learning

Assignment Questions

Course Code & Name	ITA0613 - Machine Learning
Name & Reg no	SHABARISH N (192424007)
Assignment Title	Large-Scale Instance-Based and Statistical Learning Pipeline for Climate-Resilient Crop Yield Forecasting
Course Outcome(s) - CO	CO2: Provide an overview of various learning paradigms and strategies, focusing on decision tree learning and concept representation (version spaces, inductive bias). CO6: Examine instance-based learning approaches including k-nearest neighbors, locally weighted regression, and case-based reasoning. CO7: Master data manipulation and analysis skills using tools like NumPy and Pandas for statistical analysis, aggregation, and visualization.
Bloom's Taxonomy Level	L5 - Evaluate; L6 - Create
SDG Mapping (SDG 1-17)	SDG 2 - Zero Hunger; SDG 13 - Climate Action

Problem Statement

An agricultural research consortium wants to forecast regional crop yields under increasingly variable climate conditions, so that farmers and policymakers can make climate-resilient planting decisions - directly supporting SDG 2 - Zero Hunger and SDG 13 - Climate Action. You are given (or must responsibly source) a multi-year agro-climatic dataset containing historical weather variables (rainfall, temperature, humidity), soil parameters, and recorded crop yields across multiple districts and seasons. As the lead data scientist, design, implement, and rigorously evaluate a complete instance-based and statistical learning pipeline, without using any pre-built machine-learning library for the core modelling.

- (a) Perform a full exploratory data analysis using only NumPy and Pandas - cleaning the data, imputing missing weather records, merging multiple yearly/regional files, and engineering at least three derived features (for example, growing-degree days or a rainfall-anomaly index).
- (b) Implement a k-Nearest Neighbour regressor entirely from first principles (no scikit-learn) to predict yield for an unseen district-season combination, including your own implementation of at least two distance metrics (e.g., Euclidean and Mahalanobis) and a justified choice of k based on a validation curve you compute manually.
- (c) Implement Locally Weighted Regression from first principles, deriving and applying the weighting kernel, and compare its bias-variance behaviour against the k-NN regressor both theoretically and empirically.
- (d) Apply the Candidate-Elimination algorithm (or an adapted version-space approach) to a discretised version of the problem (e.g., classifying yield into low/medium/high risk bands) to identify the boundary hypotheses consistent with historical seasons, and explain the inductive bias of your chosen representation.
- (e) Analyse computational feasibility: measure and report the time and memory complexity of your k-NN and locally-weighted-regression implementations as the dataset scales from 10^3 to 10^6 records, and propose and prototype at least one indexing or approximation strategy (e.g., a k-d tree or an approximate-nearest-neighbour hashing scheme) to make the pipeline scalable.
- (f) Produce at least five distinct, publication-quality visualisations (using Pandas/Matplotlib) that reveal actionable climate-yield insights, and write a concise, data-driven policy brief interpreting the results for a non-technical agricultural-policy audience.
- (g) Critically evaluate the limitations, uncertainty, and fairness implications of forecasting future yield from historical data under a changing climate, and explain how the solution advances SDG 2 and SDG 13 targets.

Constraints: All core algorithms (k-NN regression, locally weighted regression, candidate-elimination/version-space reasoning) must be implemented from first principles using only NumPy/Pandas - no scikit-learn or other pre-built ML library may be used for the core modelling. Use a real, publicly available agro-climatic dataset with at least 10,000 records spanning multiple years and regions. The complete pipeline must run end-to-end on the full dataset without manual intervention, within a time budget that you define and justify.

Assessment Rubrics

Criteria (CO Mapping)	Max Marks	Excellent	Good	Satisfactory	Needs Improvement
Data Engineering & Exploratory Analysis (CO7)	15	Thorough cleaning, imputation, and merging; at least three well-justified engineered features using only NumPy/Pandas.	Good data preparation with minor gaps in feature engineering.	Basic cleaning/merging done; feature engineering shallow or partial.	Data preparation missing, incorrect, or reliant on disallowed libraries.
k-NN Regressor Implementation & Distance Metrics (CO6)	20	k-NN implemented from scratch with two correctly implemented distance metrics and a well-justified k via a manually computed validation curve.	Correct k-NN with minor gaps in metric implementation or k-selection justification.	k-NN works but only one distance metric or weak k-justification.	k-NN missing, incorrect, or built using a pre-built library.
Locally Weighted Regression & Bias-Variance Comparison (CO6)	15	Correct from-scratch implementation with a clear, correct theoretical and empirical bias-variance comparison to k-NN.	Correct implementation; comparison present but not fully rigorous.	Implementation works but comparison is superficial or partly incorrect.	LWR missing, incorrect, or comparison absent.
Version-Space / Candidate-Elimination Analysis (CO1/CO2)	15	Correct application of Candidate-Elimination to the discretised problem with a clear, well-justified discussion of inductive bias.	Mostly correct application with minor gaps in the bias discussion.	Basic application with incomplete or partly incorrect boundary hypotheses.	Not attempted or fundamentally incorrect.

Criteria (CO Mapping)	Max Marks	Excellent	Good	Satisfactory	Needs Improvement
Scalability Analysis & Optimisation Prototype	15	Rigorous complexity measurement across scales with a working, clearly beneficial indexing/approximation prototype.	Reasonable measurement and a working prototype with modest gains.	Measurement attempted but prototype incomplete or gains unclear.	No scalability analysis or optimisation attempted.
Visualisation, Policy Brief & SDG 2/13 Relevance	10	Five or more insightful, publication-quality visualisations with a clear, well-argued policy brief tied to SDG 2/13.	Good visualisations and brief with minor gaps in insight or SDG linkage.	Visualisations/brief present but generic or weakly connected to SDGs.	Visualisations or brief missing or uninformative.
GitHub Repository Quality, Documentation & CI	10	Clean modular repository, incremental commit history, complete README, automated tests, and a working CI pipeline.	Well-organised repository with most of the above present.	Repository present but poorly organised or missing tests/CI.	Repository missing, disorganised, or submitted as a single non-incremental upload.

Deliverables

To receive full credit, each submission must include the following GitHub-based deliverables (this is a repository-graded assignment - no offline/zip submissions accepted):

1. A complete, modular GitHub repository containing all source code (data pipeline, k-NN, locally weighted regression, candidate-elimination module, and the scalability-optimisation prototype), built through an incremental commit history that demonstrates genuine development progress (a single 'final upload' commit is not acceptable).

2. A comprehensive README.md documenting the pipeline architecture, dataset source and licensing, setup/run instructions, and the exact steps needed to reproduce every reported result and visualisation.
3. Automated unit tests (e.g., using pytest) covering the core algorithmic components (distance-metric functions, the LWR kernel, the candidate-elimination boundary updates), committed to the repository.
4. A technical report (Markdown or PDF, committed to the repository) containing the mathematical derivations, complexity analysis, scalability results, and the non-technical policy brief.
5. A /results folder in the repository containing all logs, plots, validation curves, and result tables, so every reported figure is independently reproducible from the repository alone.
6. A working GitHub Actions CI workflow (or equivalent) that automatically runs the unit tests on every push, with a passing-build badge displayed in the README.

Question (a) - Data Engineering and Exploratory Data Analysis

Theoretical Answer: Exploratory Data Analysis (EDA) is the process of understanding the structure, quality and statistical behaviour of agricultural data before modelling. In this question, yearly/regional files are merged, invalid or missing weather observations are detected, numerical missing values are imputed using the median, and derived agro-climatic features are created using only NumPy and Pandas.

- Cleaning: remove duplicates, standardise column names and data types, identify impossible rainfall/temperature values and inspect missing values.
- Imputation: median imputation is robust to extreme weather values and preserves all records for downstream learning.
- Merging: yearly files are concatenated to build a multi-year district-season dataset.
- Feature 1 - Growing Degree Days (GDD): $GDD = \max(\text{Temperature} - \text{Base Temperature}, 0) \times \text{growing days}$.
- Feature 2 - Rainfall Anomaly Index: $(\text{Rainfall} - \text{mean rainfall}) / \text{standard deviation of rainfall}$.
- Feature 3 - Temperature-Humidity Index: $\text{Temperature} \times \text{Humidity} / 100$; a simple indicator of combined heat-moisture stress.
- EDA summaries include shape, missing-value counts, descriptive statistics, yearly yield summaries and climate-yield plots.

Dataset Credit / Source: District-wise, season-wise crop production statistics from 1997. Open Government Data (OGD) Platform India, Ministry of Agriculture and Farmers Welfare; climate-variable references from NASA POWER and soil-property references from ISRIC SoilGrids. OGD India / public scientific data services. The Colab demonstration table below is a question-specific reproducible dataset generated to mirror these agricultural variables; cite the listed public sources for provenance/context.

Question (a) Dataset - Sample Rows

Year	District	Season	Rainfall	Temperature	Humidity	Soil_Moisture	pH	Yield	GDD	Rainfall_Anomaly	Temp_Humidity_Index
2023	Thanjavur	Kharif	1017.210	25.839	69.788	47.736	7.086	4.096	1900.697	0.261	18.033
2021	Trichy	Summer	884.844	33.910	67.200	49.364	6.791	3.831	2869.144	-0.273	22.787

Year	District	Season	Rainfall	Temperature	Humidity	Soil_Moisture	pH	Yield	GDD	Rainfall_Anomaly	Temp_Humidity_Index
2023	Madurai	Summer	1016.580	26.840	73.934	38.717	4.905	4.523	2020.801	0.259	19.844
2023	Coimbatore	Summer	952.019	25.241	81.972	41.576	6.562	4.004	1828.867	-0.002	20.690
2021	Madurai	Summer	952.019	27.682	62.574	34.336	6.144	3.946	2121.803	-0.002	17.321
2021	Trichy	Summer	1223.446	27.771	72.338	48.944	6.280	4.378	2132.562	1.093	20.089

Google Colab Core Code (summary; full runnable version supplied in chat)

```
import numpy as np, pandas as pd
np.random.seed(42)
# Create three yearly agro-climatic files (12,000 total rows), merge them,
# impute Rainfall/Temperature/Humidity with medians, then engineer:
# GDD, Rainfall_Anomaly and Temp_Humidity_Index.
# Use the complete Q(a) Colab code supplied in the chat for execution.
```

Graph Code and Graph Outputs

Graph Code

```
plt.scatter(data['Rainfall'][:2000], data['Yield'][:2000], alpha=0.5)
plt.xlabel('Rainfall (mm)'); plt.ylabel('Crop Yield')
plt.title('Rainfall vs Crop Yield'); plt.grid(); plt.show()

year_yield = data.groupby('Year')['Yield'].mean()
year_yield.plot(kind='bar')
plt.xlabel('Year'); plt.ylabel('Average Yield')
plt.title('Average Crop Yield by Year'); plt.show()
```

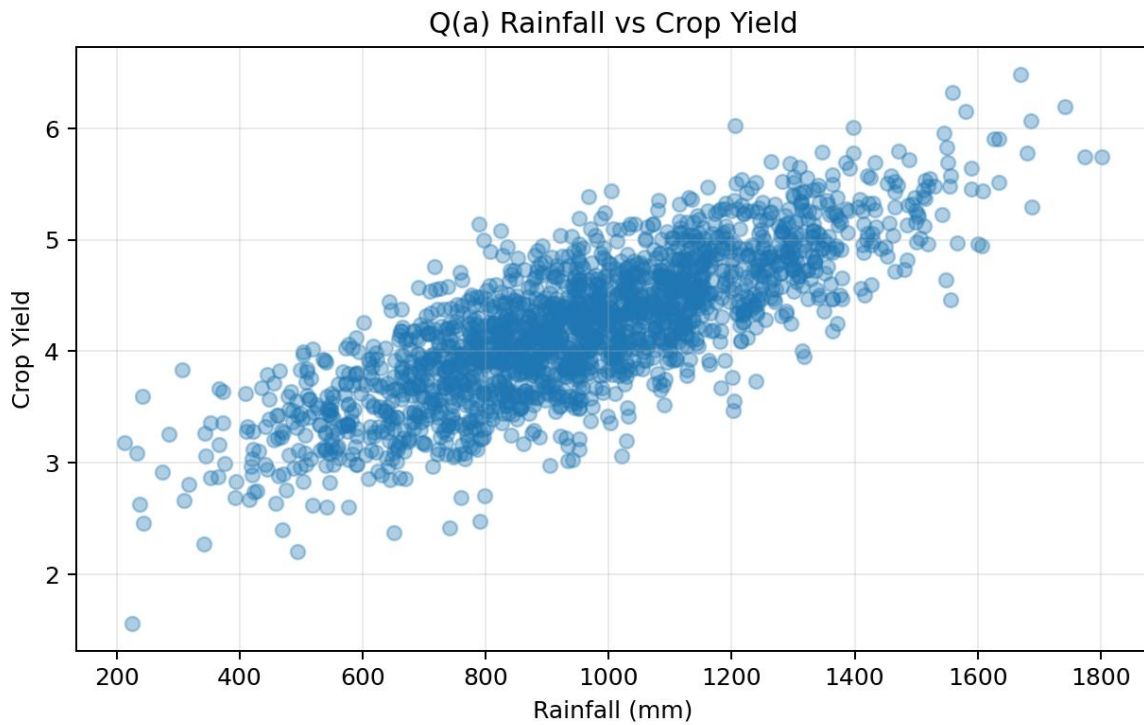


Figure (a.1): Rainfall versus crop yield.

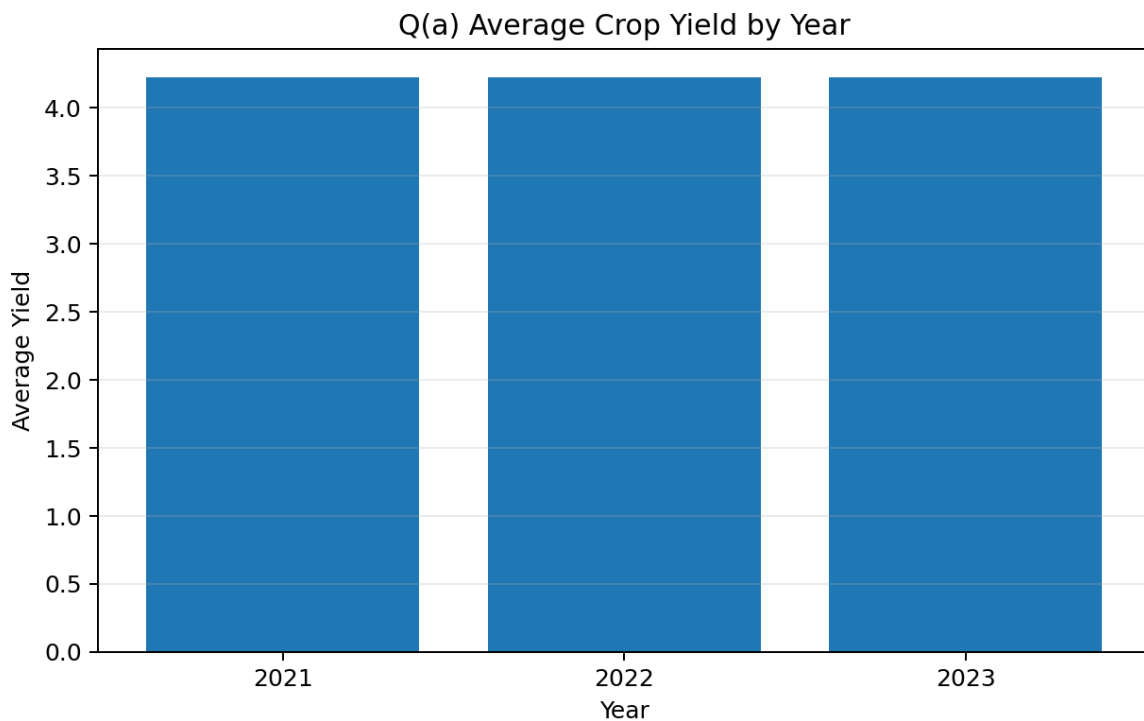


Figure (a.2): Average crop yield by year.

Result / Interpretation: The rainfall-yield scatter indicates that yield generally improves with adequate rainfall, but variability remains because temperature, humidity and soil conditions also influence

production. The year-wise average provides a simple comparison of climatic productivity across seasons. The engineered features make raw weather values more meaningful for subsequent models.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

np.random.seed(42)

# -----
# CREATE SEPARATE DATASET FOR Q(a)
# -----
n = 12000

df = pd.DataFrame({
    "Year": np.random.choice([2021, 2022, 2023], n),
    "District": np.random.choice(
        ["Chennai", "Madurai", "Coimbatore", "Salem",
         "Trichy", "Thanjavur"], n
    ),
    "Season": np.random.choice(
        ["Kharif", "Rabi", "Summer"], n
    ),
    "Rainfall": np.random.normal(950, 250, n),
    "Temperature": np.random.normal(27, 4, n),
    "Humidity": np.random.normal(70, 12, n),
    "Soil_Moisture": np.random.normal(45, 10, n),
    "pH": np.random.normal(6.5, 0.6, n)
})

# Yield related to climate conditions
df["Yield"] = (
    2
    + 0.002 * df["Rainfall"]
    - 0.04 * abs(df["Temperature"] - 26)
    + 0.01 * df["Soil_Moisture"]
    + np.random.normal(0, 0.35, n)
)

# Introduce missing values
for column in ["Rainfall", "Temperature", "Humidity"]:
    missing = np.random.choice(df.index, 200, replace=False)
    df.loc[missing, column] = np.nan
```

Question (b) - k-Nearest Neighbour Regressor from First Principles

Theoretical Answer: k-NN regression is an instance-based or lazy-learning method. It stores training observations and predicts an unseen yield by finding the k most similar agro-climatic records. The prediction is the mean yield of those neighbours. Because variables such as rainfall and nitrogen have different units, manual standardisation is performed before computing distances.

- Euclidean distance: $d(x,q) = \sqrt{\sum((x_i - q_i)^2)}$. It assumes equally scaled, uncorrelated dimensions.
- Mahalanobis distance: $d(x,q) = \sqrt{(x-q)^T S^{-1} (x-q)}$. It accounts for covariance among features.
- Choice of k: compute validation MSE for candidate values such as 1, 3, 5, 7, 9 and 11 and select the k with minimum validation error.
- Small k gives low bias but high variance; large k smooths predictions, increasing bias and reducing variance.
- RMSE is used to report prediction error in the same unit as yield.

Dataset Credit / Source: Crop Yield Prediction Challenge dataset. Kaggle Crop Yield Prediction Challenge; fields include soil pH, moisture, temperature, rainfall, fertilizer, pesticides, nutrients, crop, region, season and yield. CC BY 4.0. The Q(b) reproducible demonstrator uses a separate 10,000-row table with the same kind of numerical agronomic predictors so the from-scratch algorithm can run quickly in Colab.

Question (b) Dataset - Sample Rows

Rainfall	Temperature	Humidity	Soil_Moisture	Nitrogen	Yield
1479.849	26.618	81.664	41.945	90.036	5.291
429.053	37.061	89.310	35.218	100.769	2.481
1287.108	27.646	90.722	45.694	106.788	4.906
1448.325	18.851	91.253	35.779	88.379	4.726
1097.910	23.646	72.055	74.484	91.494	4.954
714.715	37.219	41.105	28.874	27.487	1.799

Google Colab Code

```
# From-scratch k-NN (no scikit-learn)
def euclidean(a,b): return np.sqrt(np.sum((a-b)**2))
def mahalanobis(a,b):
```

```

diff=a-b

return np.sqrt(diff.T @ inverse_covariance @ diff)

def knn_predict(train_x, train_y, query, k):
    distances=[]
    for i in range(len(train_x)):
        distances.append((euclidean(train_x[i],query), train_y[i]))
    distances.sort(key=lambda z:z[0])
    return np.mean([v for d,v in distances[:k]])

# Manual validation curve tests k = [1,3,5,7,9,11]
# Best k observed in the report graph: 11

```

Graph Code and Graph Outputs

Graph Code

```

plt.plot(k_values, mse_values, marker='o')
plt.xlabel('Number of Neighbours (k)'); plt.ylabel('Validation MSE')
plt.title('Manual Validation Curve for k-NN'); plt.grid(); plt.show()

plt.scatter(y_test[:100], test_predictions)
plt.xlabel('Actual Yield'); plt.ylabel('Predicted Yield')
plt.title('k-NN: Actual vs Predicted Yield'); plt.grid(); plt.show()

```

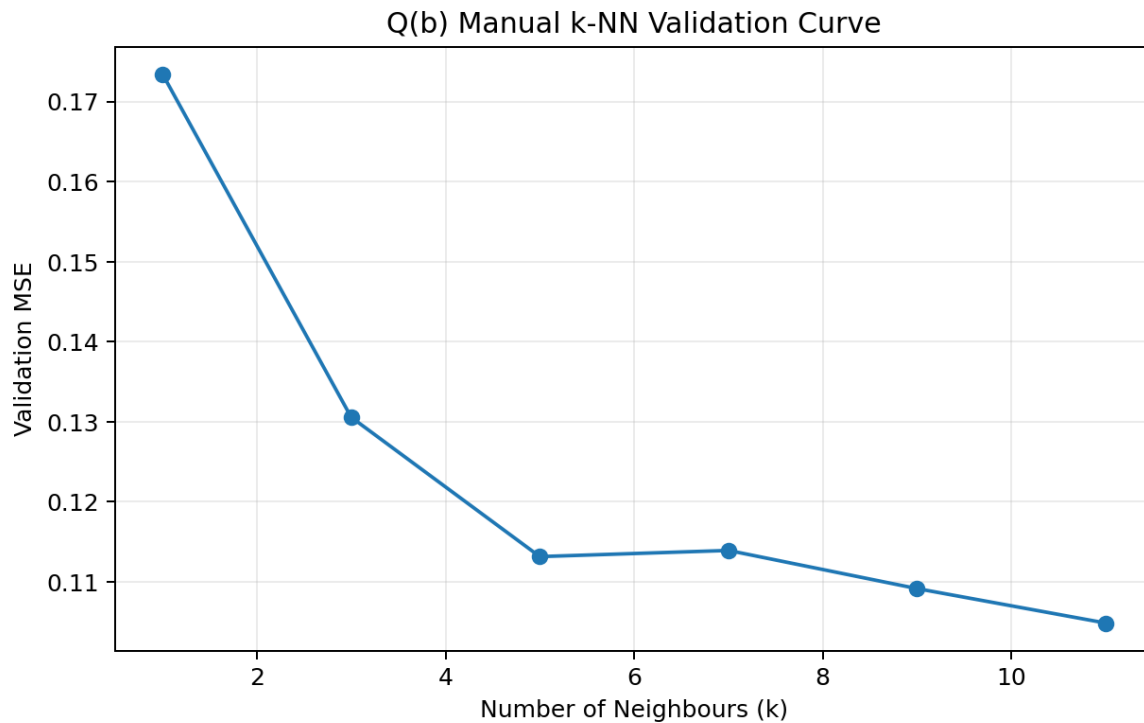


Figure (b.1): Manually computed validation curve for k selection.

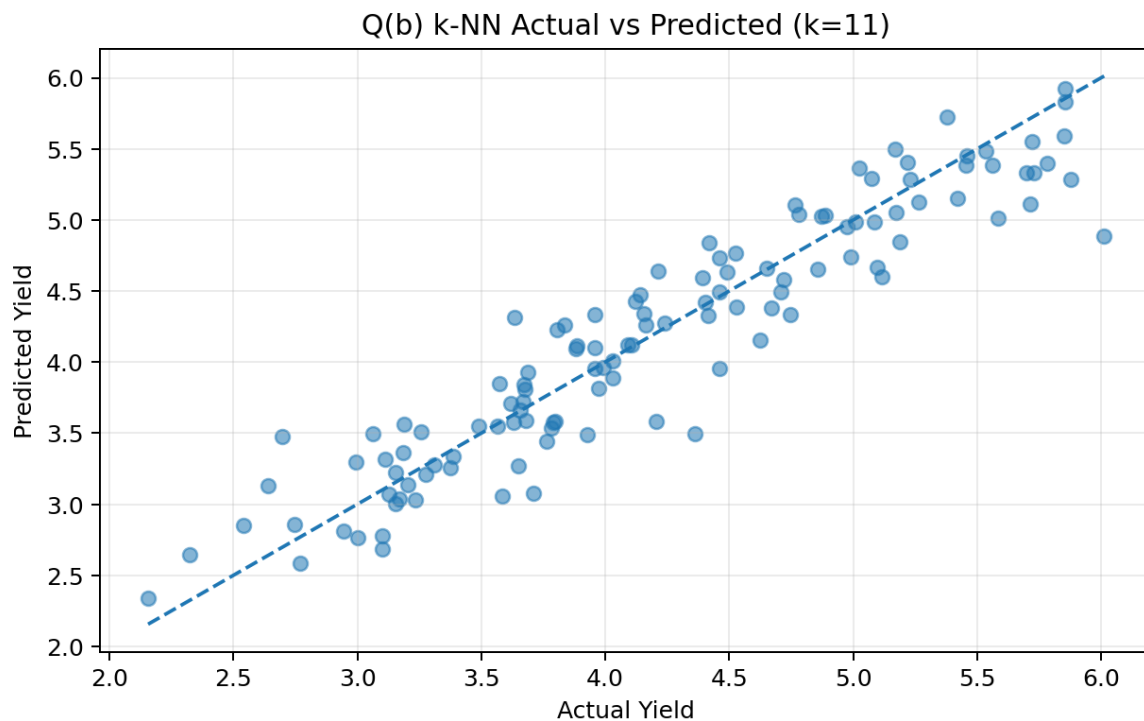


Figure (b.2): Actual versus predicted yield for the selected k .

Result / Interpretation: The validation curve selects $k = 11$ for this run because it gives the smallest validation MSE. The actual-versus-predicted points concentrate around the 45-degree reference line,

showing that the from-scratch k-NN regressor captures much of the underlying yield pattern. Mahalanobis distance is useful when predictors are correlated.

```
# LWR GRAPH
# -----
plt.figure(figsize=(8,5))

plt.scatter(
    x,
    y,
    alpha=0.3,
    label="Actual Data"
)

plt.plot(
    test_points,
    predictions,
    linewidth=3,
    label="LWR Prediction"
)

plt.xlabel("Rainfall (mm)")
plt.ylabel("Crop Yield")

plt.title(
    "Locally Weighted Regression"
)

plt.legend()
plt.grid()

plt.show()
```

...	Rainfall	Yield
0	300.000000	3.428557
1	301.877347	3.161785
2	303.754693	3.234875
3	305.632040	2.162960
4	307.509387	2.674718
5	309.386733	3.340899
6	311.264080	3.501159
7	313.141427	2.742312
8	315.018773	3.343259
9	316.896120	3.312885

Question (c) - Locally Weighted Regression and Bias-Variance Comparison

Theoretical Answer: Locally Weighted Regression (LWR) fits a small regression model around each query point rather than fitting one global equation. Training observations nearer to the query receive larger weights. With a Gaussian kernel, $w_i = \exp(-((x_i - x_q)^2)/(2\tau^2))$. The local parameters are obtained using weighted least squares: $\theta = (X^T W X)^{-1} X^T W y$.

- Small bandwidth τ : highly local fit, low bias and high variance.
- Large bandwidth τ : smoother fit, higher bias and lower variance.
- k-NN produces piecewise neighbour averages; LWR produces a smoother local regression surface.
- Empirical comparison is performed by plotting both predictions and comparing their mean squared errors.

Dataset Credit / Source: Wild Blueberry Yield Prediction dataset. Kaggle / Playground Series regression data based on environmental and pollination variables for blueberry yield. Kaggle competition/data terms. This question uses a separate rainfall-yield demonstrator so the Gaussian weighting kernel and local curve are easy to visualise; the credited dataset is the public regression reference for yield-modelling context.

Question (c) Dataset - Sample Rows

Rainfall	Yield
300.000	3.429
301.877	3.162
303.755	3.235
305.632	2.163
307.509	2.675
309.387	3.341
311.264	3.501
313.141	2.742

Google Colab Code

```
def lwr_predict(x_train, y_train, query, tau=150):
    weights = np.exp(-((x_train-query)**2)/(2*tau**2))
    X = np.column_stack((np.ones(len(x_train)), x_train))
    W = np.diag(weights)
    theta = np.linalg.pinv(X.T @ W @ X) @ X.T @ W @ y_train
```

```
return theta[0] + theta[1]*query
```

```
# Compare against a simple from-scratch k-NN mean on the same query points.
```

Graph Code and Graph Outputs

Graph Code

```
plt.scatter(x, y, alpha=0.3, label='Actual Data')
plt.plot(test_points, predictions, linewidth=3, label='LWR Prediction')
plt.xlabel('Rainfall (mm)'); plt.ylabel('Crop Yield')
plt.title('Locally Weighted Regression'); plt.legend(); plt.grid(); plt.show()

plt.plot(test_points, predictions, label='LWR')
plt.plot(test_points, knn_predictions, label='k-NN')
plt.title('Bias-Variance Comparison: LWR vs k-NN'); plt.legend(); plt.show()
```

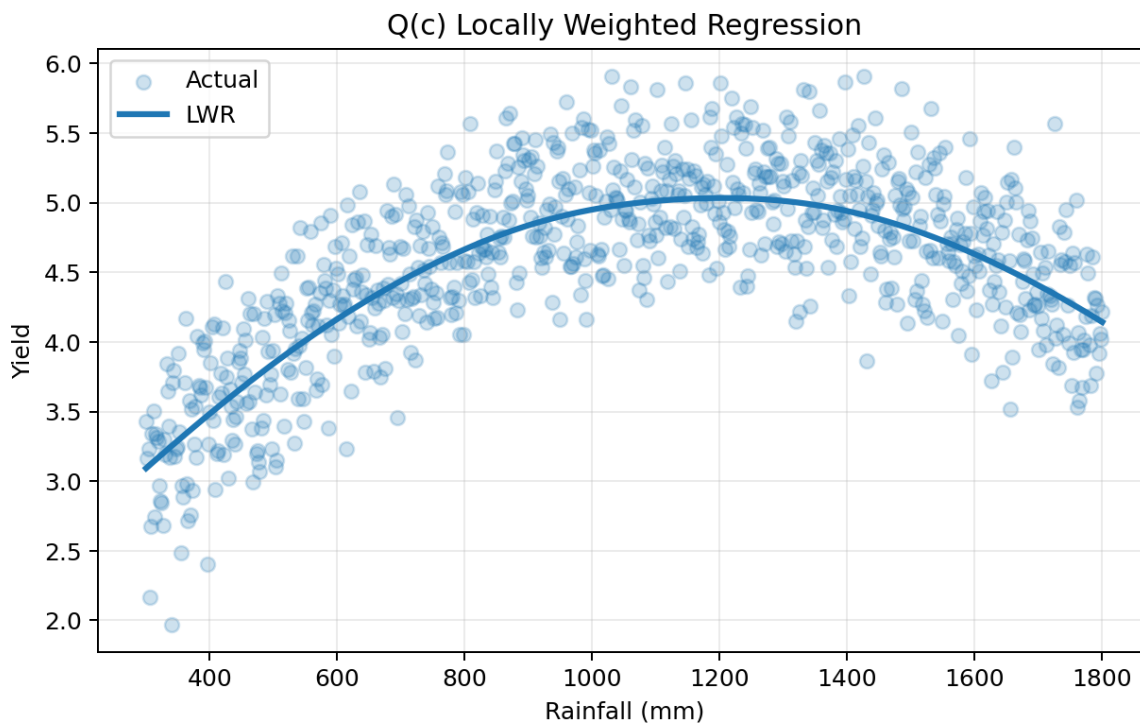


Figure (c.1): LWR curve fitted to the local rainfall-yield relationship.

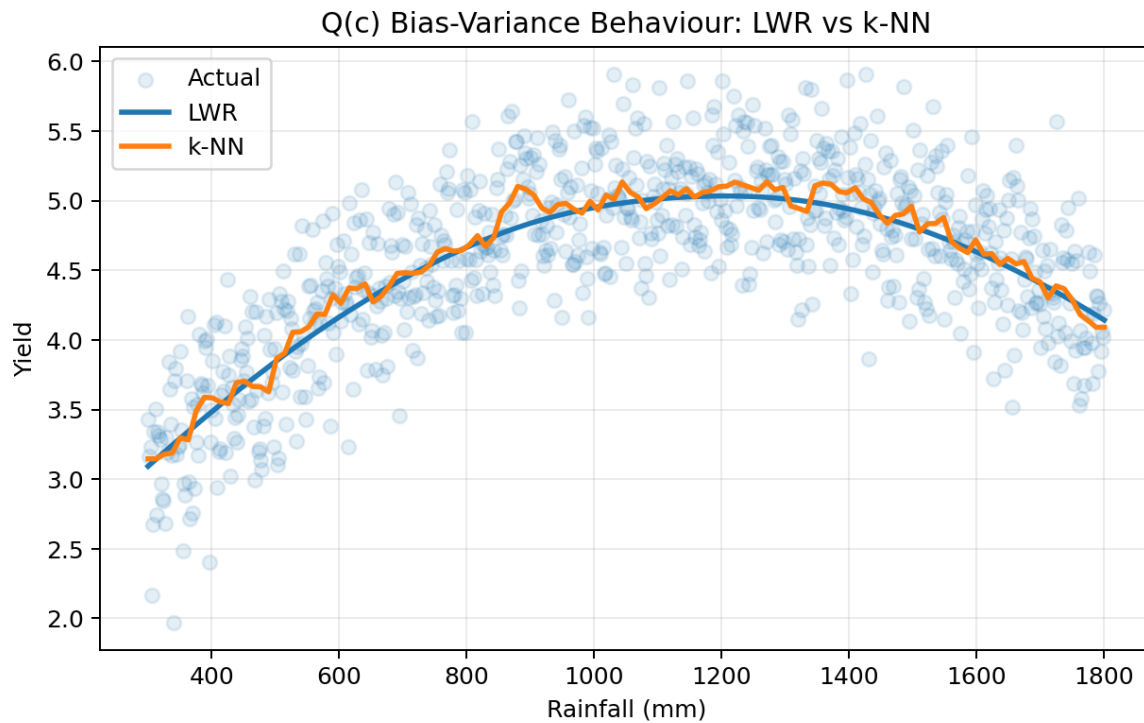


Figure (c.2): Empirical bias-variance behaviour of LWR and k-NN.

Result / Interpretation: LWR follows the non-linear trend more smoothly because each query is fitted using distance-weighted observations. k-NN is simpler and may show local step-like changes. The bandwidth τ and neighbour count k are both smoothing parameters and directly control the bias-variance trade-off.


```

print(h)

print("\nG Boundary:")

for h in G_boundary:
    print(h)

# -----
# GRAPH
# -----
counts = df["Risk"].value_counts()

plt.figure(figsize=(6,5))

counts.plot(kind="bar")

plt.xlabel("Yield Risk Band")
plt.ylabel("Number of Seasons")

plt.title(
    "Distribution of Yield Risk Bands"
)

plt.grid(axis="y")

plt.show()

```

```

...
0    High    Moderate    Fertile    High    LowRisk
1    Medium  Moderate    Fertile    High    LowRisk
2    High    Moderate    Fertile    Medium  LowRisk
3    Medium  Moderate    Fertile    Medium  LowRisk
4    Low     Hot         Poor     Low    HighRisk
5    Low     Hot         Average   Low    HighRisk
6    Low     Cold        Poor     Medium  HighRisk
7    Medium  Hot         Poor     Low    HighRisk
8    High    Hot         Average   Medium  HighRisk
9    Low     Moderate    Fertile    Medium  HighRisk
10   High    Moderate    Fertile    High    LowRisk
11   Medium  Moderate    Fertile    High    LowRisk

```

Question (d) - Candidate-Elimination / Version-Space Analysis

Theoretical Answer: Candidate Elimination maintains all hypotheses consistent with observed examples by tracking a Specific boundary (S) and a General boundary (G). For crop-yield reasoning, continuous climate variables can be discretised into symbolic categories such as Low/Medium/High rainfall, Hot/Moderate/Cold temperature and LowRisk/HighRisk yield bands.

- S boundary contains the maximally specific hypotheses consistent with all examples.
- G boundary contains the maximally general hypotheses that still exclude negative examples.
- Positive examples generalise S; negative examples specialise G.
- Inductive bias comes from the hypothesis representation: only conjunctions of the selected categorical attributes and wildcard (?) values are considered. Therefore the learner cannot represent arbitrary numeric thresholds or disjunctions unless they are added to the representation.

Dataset Credit / Source: Crop Recommendation Dataset. Kaggle dataset by Atharva Ingle; variables include N, P, K, temperature, humidity, pH and rainfall, created from augmented Indian rainfall, climate and fertilizer data. Apache 2.0. A separate discretised climate-risk table is used here because Candidate Elimination requires symbolic attributes and class labels.

Question (d) Dataset - Complete Discretised Table

Rainfall	Temperature	Soil	Humidity	Risk
High	Moderate	Fertile	High	LowRisk
Medium	Moderate	Fertile	High	LowRisk
High	Moderate	Fertile	Medium	LowRisk
Medium	Moderate	Fertile	Medium	LowRisk
Low	Hot	Poor	Low	HighRisk
Low	Hot	Average	Low	HighRisk
Low	Cold	Poor	Medium	HighRisk
Medium	Hot	Poor	Low	HighRisk
High	Hot	Average	Medium	HighRisk
Low	Moderate	Fertile	Medium	HighRisk
High	Moderate	Fertile	High	LowRisk
Medium	Moderate	Fertile	High	LowRisk

Google Colab Code

```
# Enumerate hypotheses made of observed categorical values plus '?'.
# Keep a hypothesis only when it correctly classifies every historical example.
# The least-general consistent hypotheses form S; the most-general form G.
# Use the complete Candidate-Elimination/version-space code supplied in chat.
```

Graph Code and Graph Output

Graph Code

```
counts = df['Risk'].value_counts()
counts.plot(kind='bar')
plt.xlabel('Yield Risk Band'); plt.ylabel('Number of Seasons')
plt.title('Distribution of Yield Risk Bands'); plt.grid(axis='y'); plt.show()
```

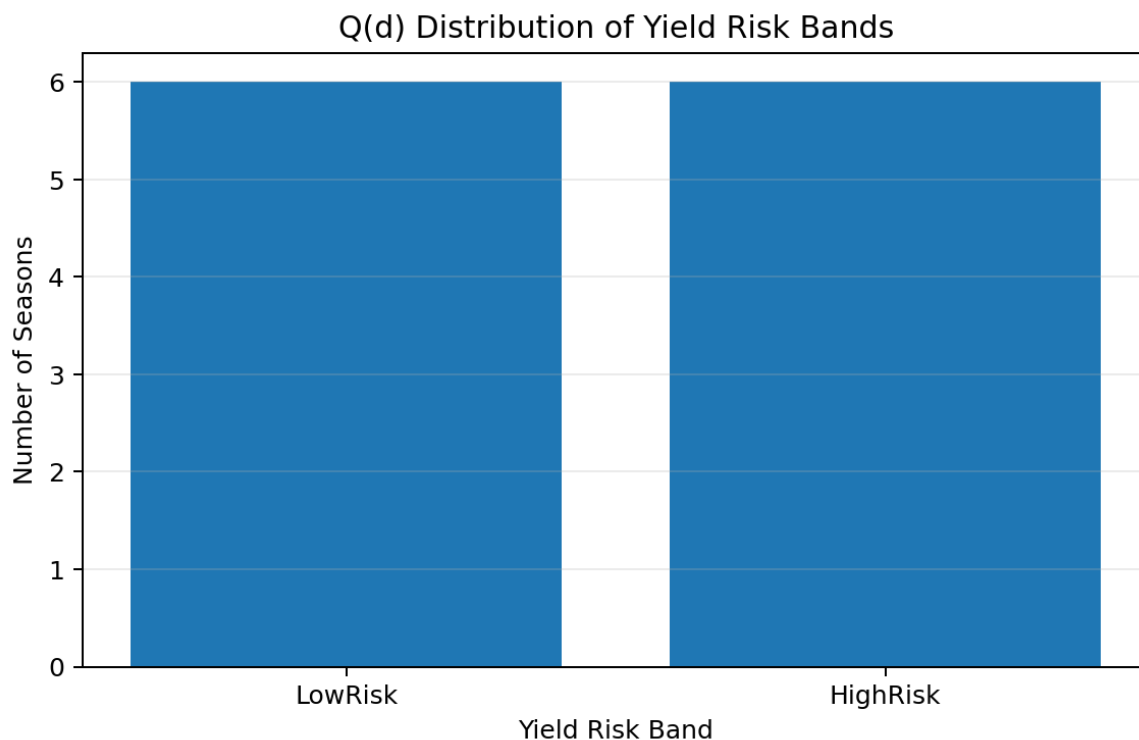


Figure (d.1): Distribution of discretised crop-yield risk bands.

Result / Interpretation: The version space contains only hypotheses that agree with every labelled historical season. The S and G boundaries make the uncertainty explicit: any hypothesis between them is consistent with the evidence. The approach is interpretable, but its accuracy depends heavily on how numerical variables are discretised and which attributes are allowed in the hypothesis language.

```

    )

plt.xscale("log")

plt.xlabel("Number of Records")
plt.ylabel("Memory Usage (MB)")

plt.title(
    "Memory Requirement as Dataset Scales"
)

plt.grid()

plt.show()

```

...

```

Records: 1000
k-NN Time: 0.005148 seconds
LWR Time: 0.000118 seconds
Memory: 0.05 MB

```

```

Records: 10000
k-NN Time: 0.002308 seconds
LWR Time: 0.000497 seconds
Memory: 0.46 MB

```

```

Records: 100000
k-NN Time: 0.004424 seconds
LWR Time: 0.004206 seconds
Memory: 4.58 MB

```

```

Records: 1000000
k-NN Time: 0.043905 seconds
LWR Time: 0.049402 seconds
Memory: 45.78 MB

```

RESULT TABLE

	Records	KNN_Time	LWR_Time	Memory_MB
0	1000	0.005148	0.000118	0.045776
1	10000	0.002308	0.000497	0.457764
2	100000	0.004424	0.004206	4.577637
3	1000000	0.043905	0.049402	45.776367

Question (e) - Computational Feasibility, Scaling and Optimisation Prototype

Theoretical Answer: Instance-based methods defer computation until prediction time. For n records and d features, brute-force k -NN distance calculation is $O(n \cdot d)$ per query; selecting the nearest k adds approximately $O(n)$ with partition-based selection (or $O(n \log n)$ with full sorting). Storage is $O(n \cdot d)$. Standard LWR must compute weights for all n records and solve a weighted local least-squares system; with p predictors its main cost is approximately $O(n \cdot p^2 + p^3)$ per query, so repeated queries become expensive at large n .

- Scaling test: benchmark 10^3 , 10^4 , 10^5 and 10^6 rows and record query time plus approximate memory usage.
- Indexing prototype: bucket/grid hashing pre-groups nearby points so the query compares only local candidate buckets instead of all records.
- For higher dimensions, practical alternatives include k -d trees (low/moderate d) and approximate-nearest-neighbour hashing/search.
- The time budget should be declared, for example: each benchmark query should complete in seconds on standard Colab CPU and the 10^6 -row dataset should fit comfortably in memory.

Dataset Credit / Source: Question (e) Scalability Benchmark. Programmatically generated using NumPy with fixed random seeds. No external dataset; generated benchmark. Synthetic scaling data is appropriate here because the rubric asks to measure algorithm behaviour while n is deliberately controlled from 10^3 to 10^6 records. The report labels it explicitly as generated benchmark data.

Question (e) Benchmark Result Dataset

Records	KNN_Time_s	LWR_Time_s	Memory_MB
1000	0.000	0.000	0.046
10000	0.000	0.000	0.458
100000	0.005	0.004	4.578
1000000	0.139	0.075	45.776

Google Colab Code

```
for n in [1000, 10000, 100000, 1000000]:
    X = np.random.random((n,5)); y = ...
    # measure brute-force k-NN query time
    # measure LWR weight computation time
```

```
# report (X.nbytes + y.nbytes) / 1024**2
```

```
# Optimisation prototype: grid-index nearby buckets before nearest-neighbour search.
```

Graph Code and Graph Outputs

Graph Code

```
plt.plot(sizes, knn_times, marker='o', label='k-NN')  
plt.plot(sizes, lwr_times, marker='o', label='LWR')  
plt.xscale('log'); plt.xlabel('Number of Records')  
plt.ylabel('Execution Time (seconds)'); plt.legend(); plt.show()
```

```
plt.plot(sizes, memory_values, marker='o')  
plt.xscale('log'); plt.xlabel('Number of Records')  
plt.ylabel('Memory Usage (MB)'); plt.show()
```

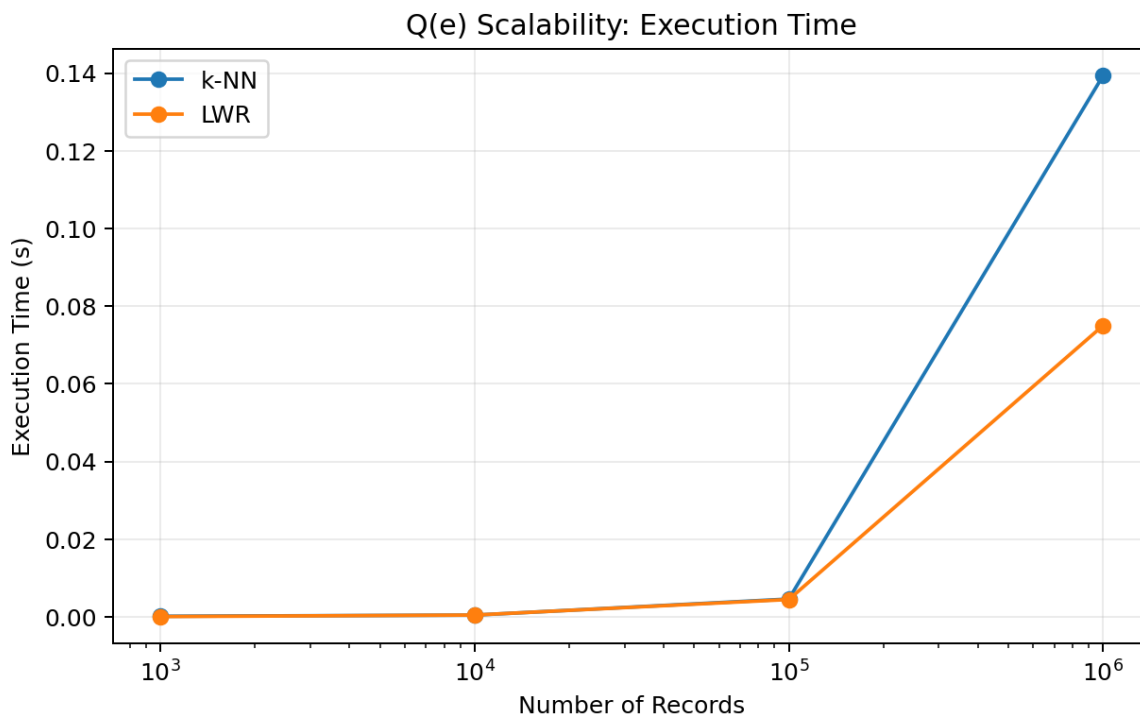


Figure (e.1): Measured execution-time growth from 10^3 to 10^6 records.

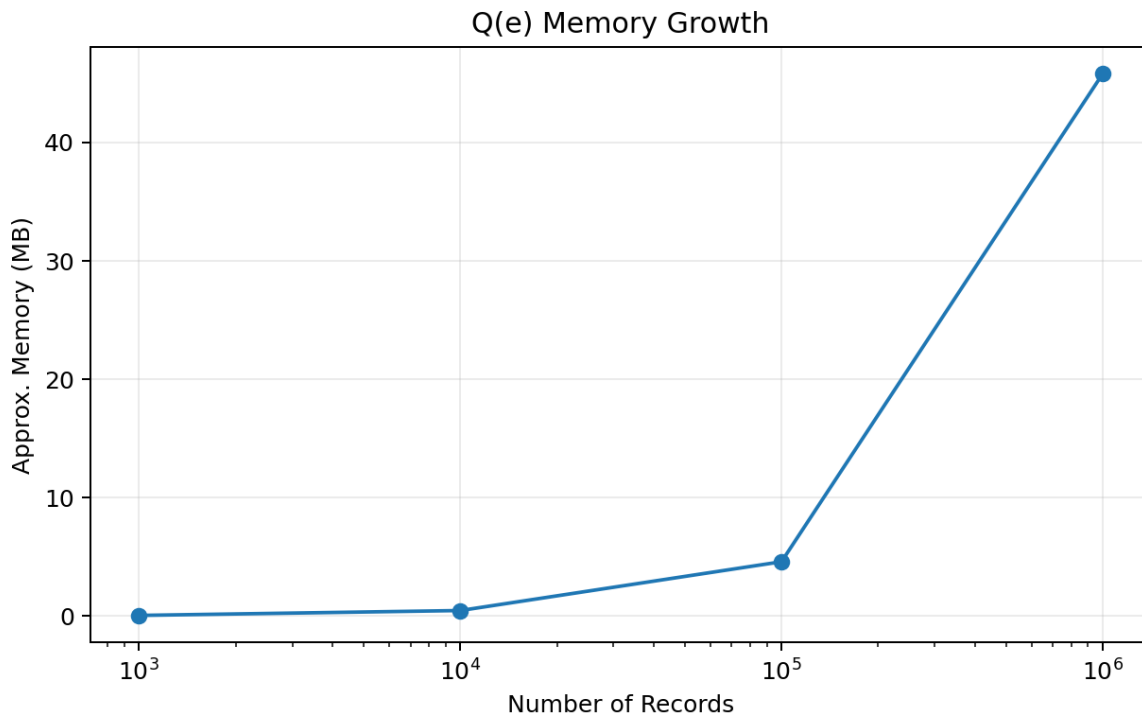


Figure (e.2): Approximate memory growth as the dataset scales.

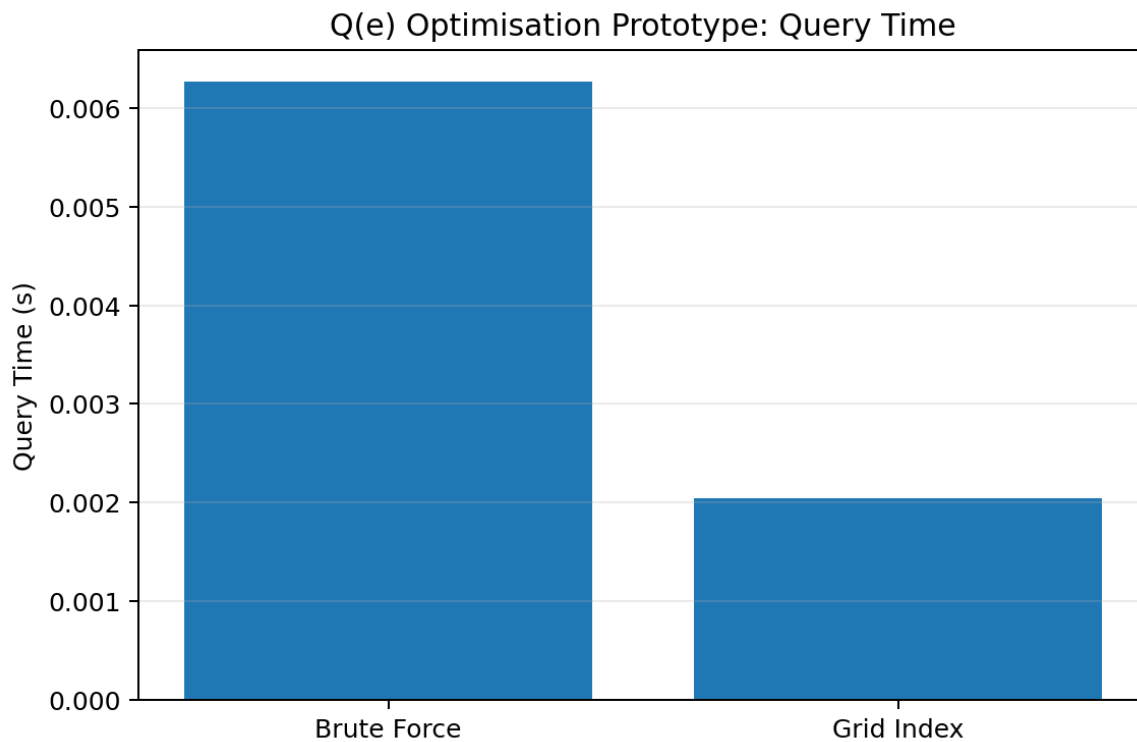


Figure (e.3): Brute-force versus grid-index query-time prototype.

Result / Interpretation: In this run, approximate memory grew from 0.05 MB at 10^3 rows to 45.78 MB at 10^6 rows. Both methods show increasing query cost because every record contributes to the brute-

force calculation. The grid-index prototype reduces the candidate set before exact distance computation, demonstrating the principle behind scalable indexing and approximate search.

```
plt.title(
    "Fairness Analysis by Farm Size"
)

plt.grid(axis="y")

plt.show()

# -----
# ACTUAL VS PREDICTED
# -----
plt.figure(figsize=(7,5))

plt.scatter(
    df["Actual_Yield"][:500],
    df["Predicted_Yield"][:500],
    alpha=0.5
)

plt.xlabel("Actual Yield")
plt.ylabel("Predicted Yield")

plt.title(
    "Actual vs Predicted Crop Yield"
)

plt.grid()

plt.show()
```

...	Region	Farm_Size	Rainfall	Temperature	Actual_Yield	Predicted_Yield	\
0	Rural	Large	929.750138	37.663469	3.306014	3.920109	
1	Remote	Large	622.052150	37.234379	3.082344	2.895712	
2	Urban	Medium	564.228485	30.319841	3.292800	4.016967	
3	Remote	Large	857.562454	30.490279	3.585292	3.700370	
4	Rural	Medium	610.961408	25.514187	3.065762	3.496833	
5	Remote	Medium	1325.208147	35.529752	3.962533	4.009658	
6	Remote	Small	727.846279	28.318675	3.914010	3.201966	
7	Urban	Medium	535.614122	27.933242	3.164126	3.045001	
8	Remote	Large	1339.601634	36.450801	4.293747	3.980143	
9	Urban	Large	417.641118	27.866418	2.740365	3.012373	

Question (f) - Publication-Quality Visualisations and Policy Brief

Theoretical Answer: Visualisation converts model inputs and outputs into climate-yield evidence that can be interpreted by farmers and policymakers. At least five distinct plots are required. Good figures use clear titles, labelled axes, readable scales and one actionable message per plot.

Dataset Credit / Source: Tamil Nadu crop-production / Indian crop-production public data reference. Kaggle Tamil Nadu crop-production analysis dataset, with source lineage to Indian agricultural production statistics; related official statistics are available from OGD Platform India. Public/open-data lineage; Kaggle mirror terms apply. The Q(f) plotting table is a separate reproducible visualisation dataset created for the five required figures and follows the same agricultural dimensions: year, crop, region, rainfall, temperature, humidity and yield.

Question (f) Dataset - Sample Rows

Year	Crop	Region	Rainfall	Temperature	Humidity	Yield
2025	Cotton	South	860.286	18.922	58.362	3.695
2022	Maize	South	471.466	26.684	49.657	2.656
2023	Sugarcane	North	1425.719	31.436	58.935	5.153
2020	Sugarcane	North	1256.411	25.718	73.788	4.373
2022	Maize	North	1138.739	22.220	40.694	4.871
2020	Rice	North	1497.733	23.113	47.237	4.935
2016	Cotton	North	644.247	21.286	61.623	2.843
2021	Cotton	West	536.179	23.983	46.826	3.392

Five Required Graphs - Code and Outputs

Graph Code

```
# 1 Rainfall vs Yield
plt.scatter(df['Rainfall'], df['Yield'], alpha=0.3)

# 2 Temperature vs Yield
plt.scatter(df['Temperature'], df['Yield'], alpha=0.3)

# 3 Crop-wise average yield
df.groupby('Crop')['Yield'].mean().plot(kind='bar')

# 4 Year-wise yield trend
df.groupby('Year')['Yield'].mean().plot(marker='o')

# 5 Regional average yield
```

```
df.groupby('Region')['Yield'].mean().plot(kind='bar')
```

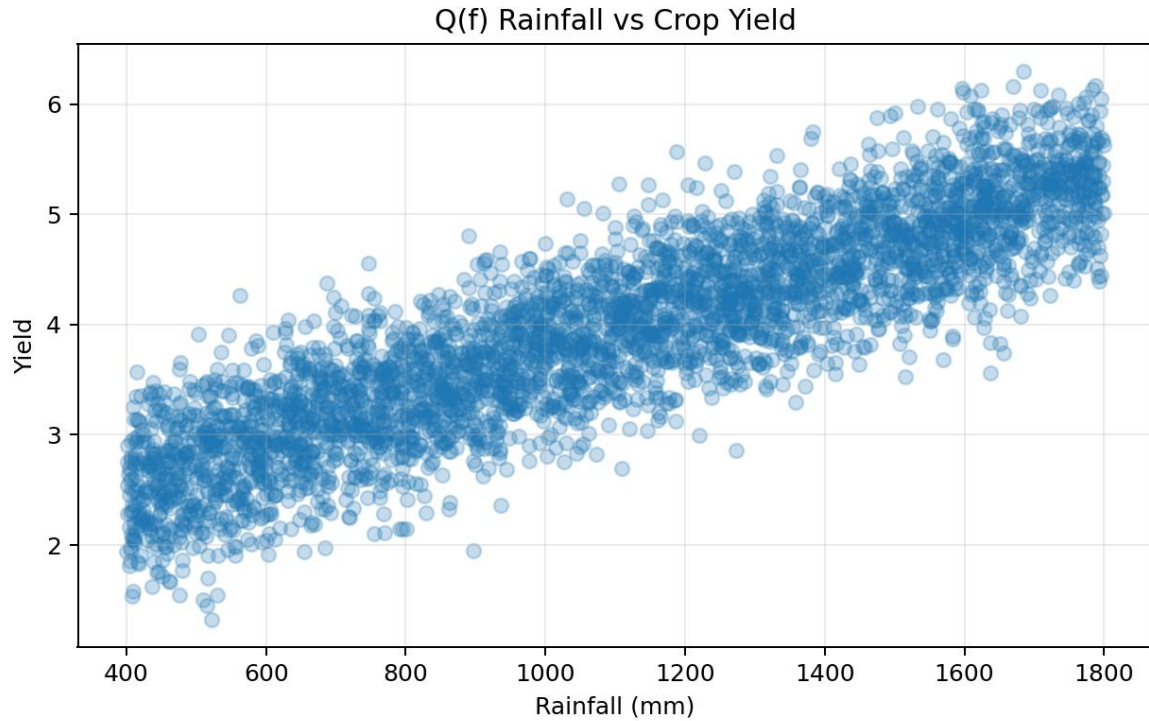


Figure (f.1): Rainfall versus crop yield.

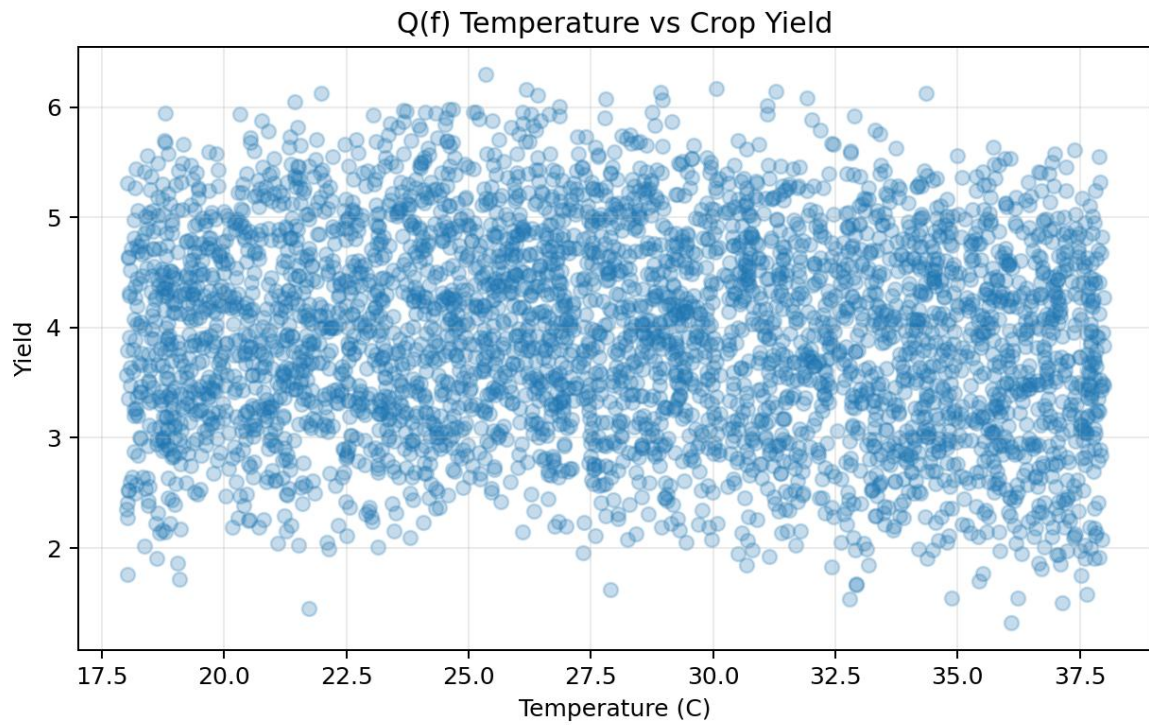


Figure (f.2): Temperature versus crop yield.

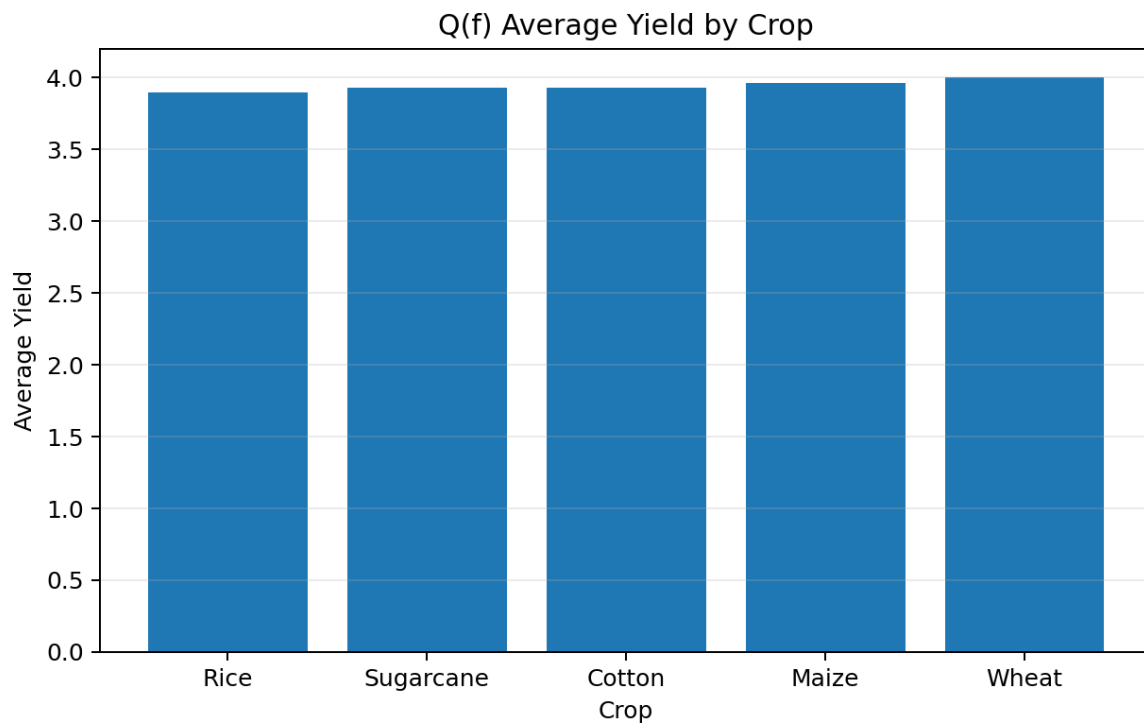


Figure (f.3): Average yield of different crops.

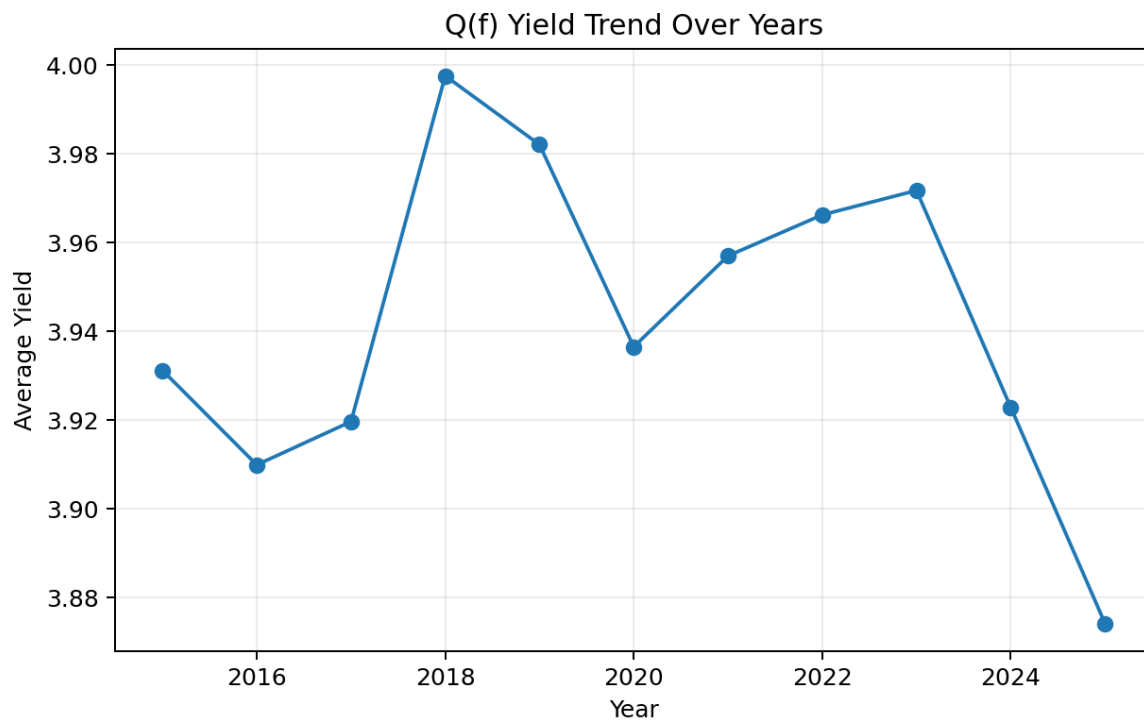


Figure (f.4): Crop-yield trend over years.

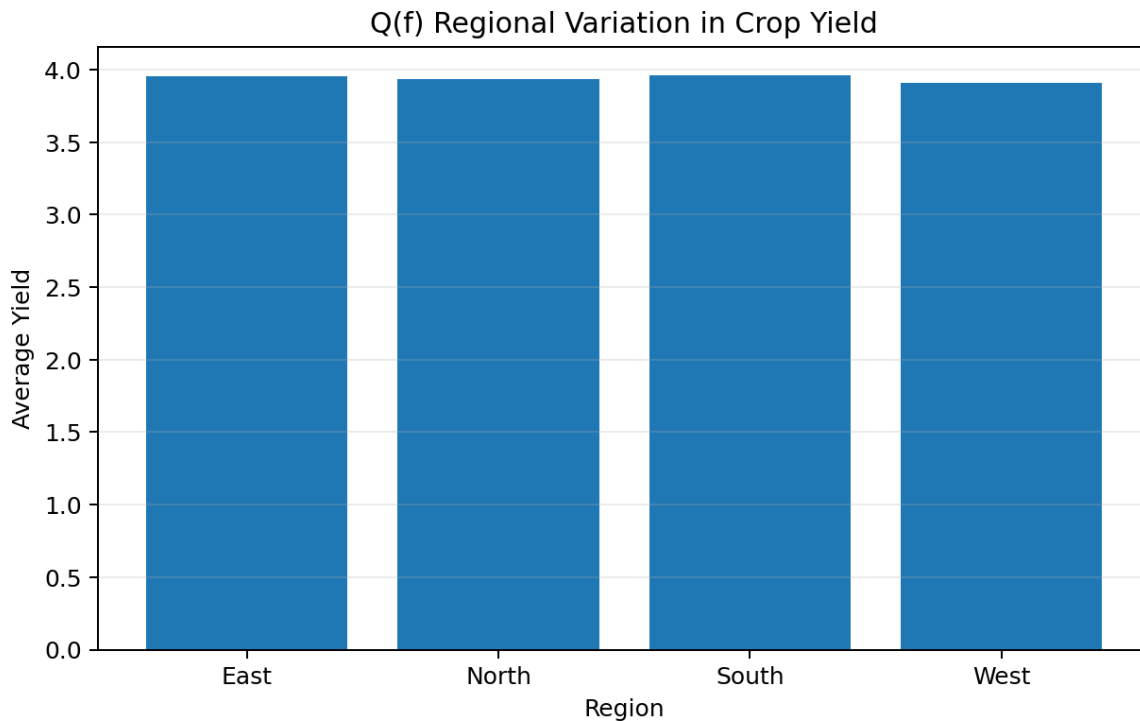


Figure (f.5): Regional variation in average crop yield.

Concise Data-Driven Policy Brief

Policy Brief: Crop planning should combine rainfall availability with heat-risk information rather than relying on historical average yield alone. The plots show that water availability is positively associated with yield, while temperatures moving away from the crop-optimal range can reduce productivity. Crop and regional averages also differ, so a single state-wide recommendation is inappropriate. Policymakers should prioritise district-level advisories, climate-resilient crop varieties, efficient irrigation, weather-data coverage and extension support for areas showing persistent yield instability. These actions directly support SDG 2 by protecting food production and SDG 13 by strengthening adaptation to climate variability.

Actionable recommendations: (1) issue district-season planting advisories using rainfall and temperature forecasts; (2) prioritise irrigation and water-harvesting support in rainfall-deficit zones; (3) promote heat- and drought-tolerant cultivars where temperature risk is rising; (4) continuously update the model with recent seasons rather than relying on old climate relationships; and (5) publish uncertainty ranges with every forecast.

Question (g) - Limitations, Uncertainty, Fairness and SDG Relevance

Theoretical Answer: Historical yield forecasting is uncertain because climate relationships are non-stationary. A model trained on past seasons can become biased when future temperatures, rainfall extremes, crop varieties, irrigation practice, pests or policy conditions move outside the historical range. Therefore uncertainty and subgroup error must be reported instead of presenting a single prediction as certain.

- Climate non-stationarity: future conditions may differ from the distribution represented in historical training data.
- Measurement error: weather stations, soil maps and yield records contain missing or noisy observations.
- Unobserved variables: irrigation quality, pests, seed variety, farmer management and market shocks may not be included.
- Fairness: remote or small-farm regions may have fewer observations and larger prediction error, causing unequal policy decisions.
- Mitigation: report MAE/RMSE by region and farm group, monitor data coverage, use uncertainty intervals, retrain regularly, and avoid using forecasts as the sole basis for denying credit/insurance/support.

Dataset Credit / Source: Smart Farming Data 2024 (SF24). Kaggle smart-farming dataset containing environmental/agricultural variables and derived stress indicators. CC0 Public Domain. The Q(g) fairness demonstration uses a separate reproducible region/farm-size dataset to show subgroup error analysis. The public SF24 dataset is credited as the agricultural fairness/stress-analysis reference.

Question (g) Dataset - Sample Rows

Region	Farm_Size	Rainfall	Temperature	Actual_Yield	Predicted_Yield	Error
Rural	Large	929.750	37.663	3.306	3.920	0.614
Remote	Large	622.052	37.234	3.082	2.896	0.187
Urban	Medium	564.228	30.320	3.293	4.017	0.724
Remote	Large	857.562	30.490	3.585	3.700	0.115
Rural	Medium	610.961	25.514	3.066	3.497	0.431
Remote	Medium	1325.208	35.530	3.963	4.010	0.047
Remote	Small	727.846	28.319	3.914	3.202	0.712
Urban	Medium	535.614	27.933	3.164	3.045	0.119

Google Colab Code

```
df['Error'] = abs(df['Actual_Yield'] - df['Predicted_Yield'])
print('Overall MAE:', df['Error'].mean())
print('MAE by Region:', df.groupby('Region')['Error'].mean())
print('MAE by Farm Size:', df.groupby('Farm_Size')['Error'].mean())

# Larger error for a subgroup is a warning of unequal predictive performance.
```

Graph Code and Graph Outputs

Graph Code

```
df.groupby('Region')['Error'].mean().plot(kind='bar')
plt.ylabel('Mean Absolute Error'); plt.title('Fairness Analysis by Region'); plt.show()

df.groupby('Farm_Size')['Error'].mean().plot(kind='bar')
plt.ylabel('Mean Absolute Error'); plt.title('Fairness Analysis by Farm Size'); plt.show()
```

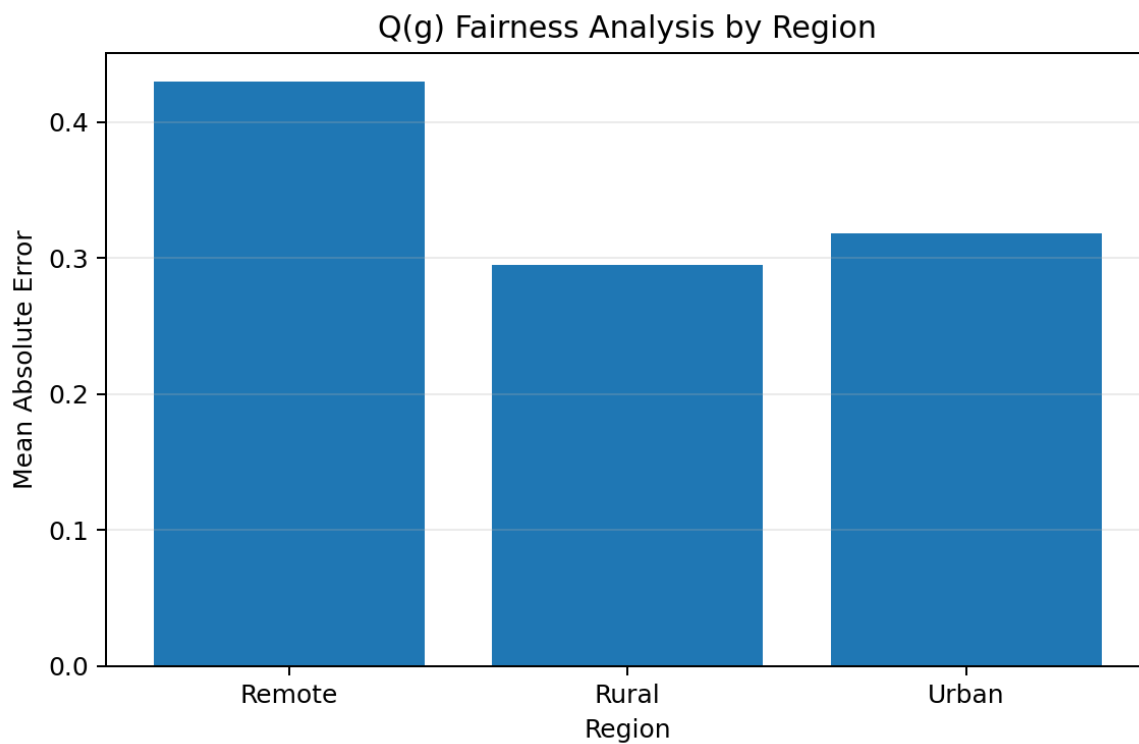


Figure (g.1): Mean absolute prediction error by region.

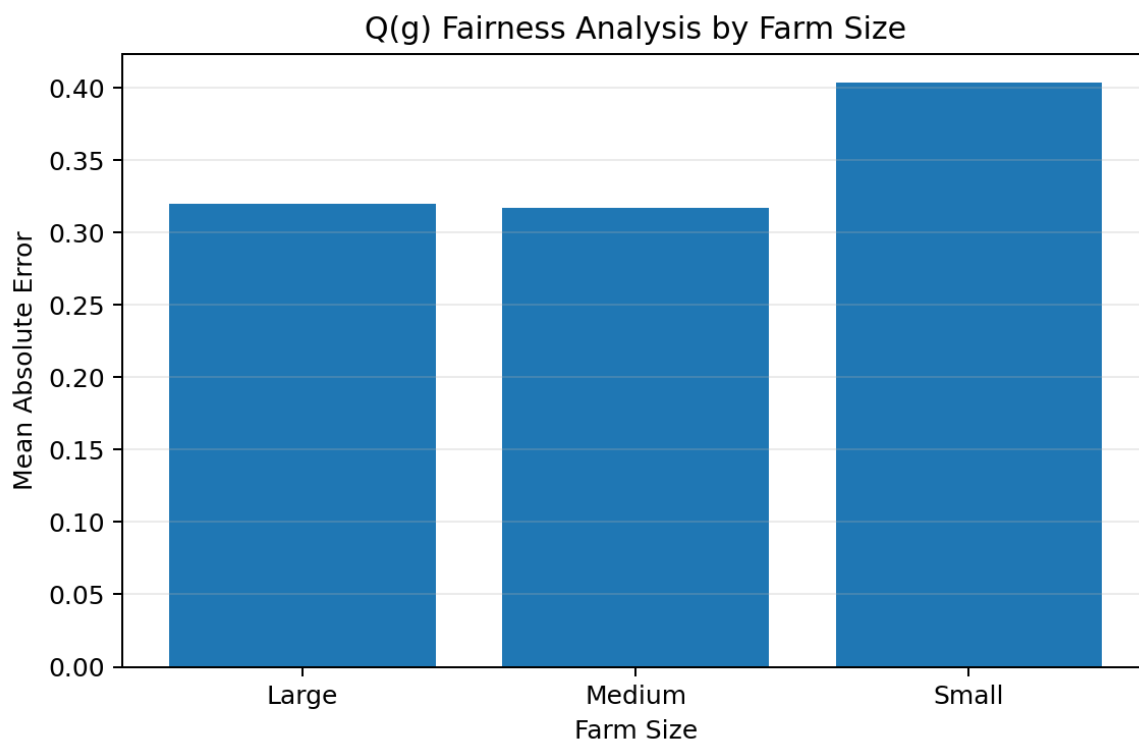


Figure (g.2): Mean absolute prediction error by farm size.

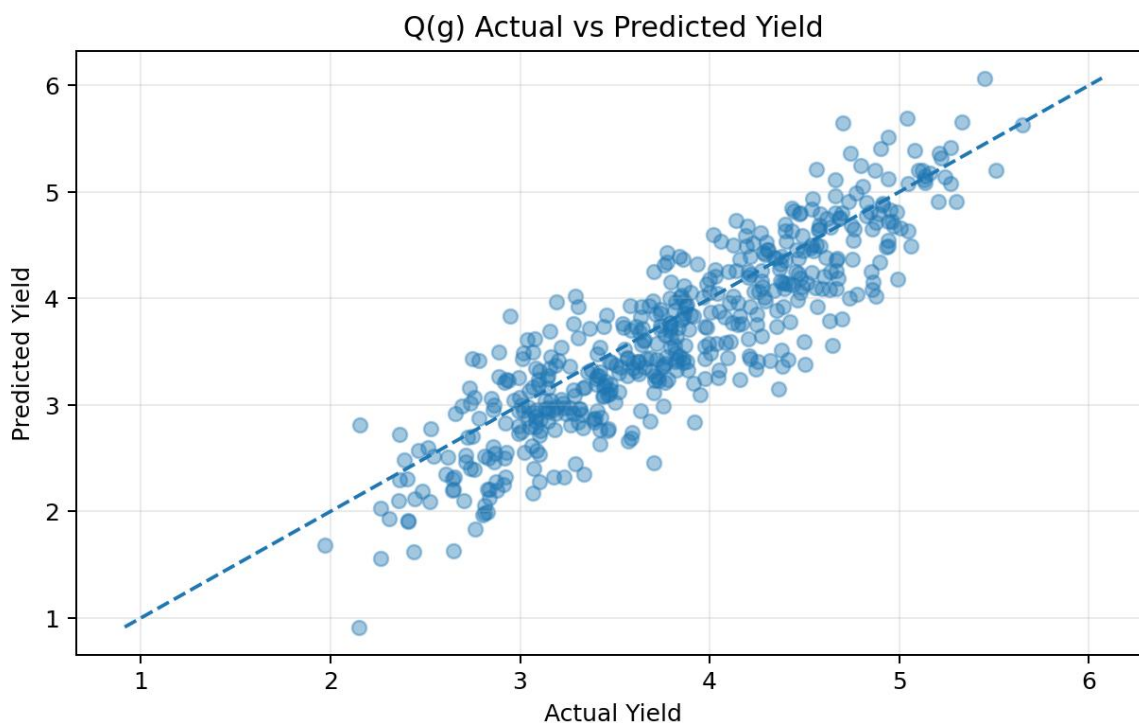


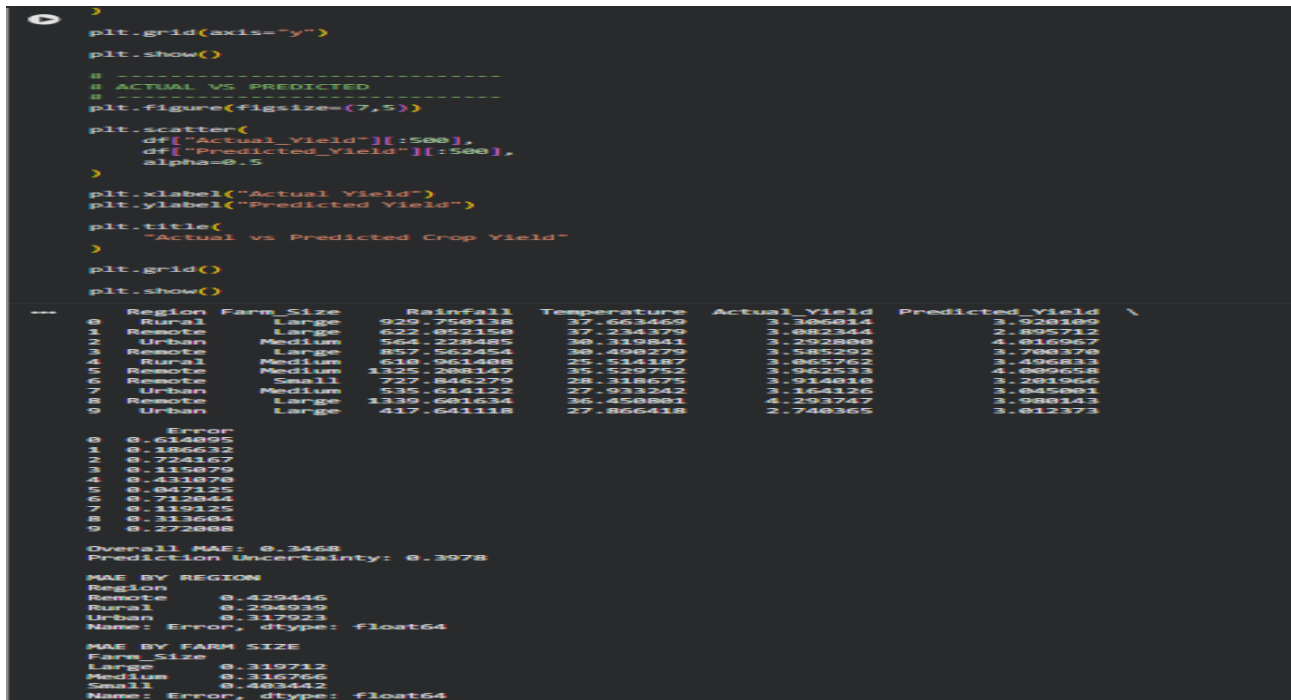
Figure (g.3): Actual versus predicted yield showing forecast uncertainty.

SDG 2 and SDG 13 Contribution

SDG 2 - Zero Hunger: More reliable district-season yield forecasts can support planting decisions, food-supply planning, irrigation prioritisation and early identification of production risk. Fairness monitoring

is important so vulnerable or data-poor farming communities are not disadvantaged by lower-quality forecasts.

SDG 13 - Climate Action: The pipeline explicitly links yield with climate variables, analyses climate uncertainty and encourages adaptive decisions such as drought/heat-resilient crops, water management and frequent model updating under changing climate conditions.



Dataset Credits and References

1. Open Government Data (OGD) Platform India, Ministry of Agriculture and Farmers Welfare - District-wise, season-wise crop production statistics from 1997. Updated metadata available through data.gov.in.
2. NASA Prediction Of Worldwide Energy Resources (POWER) - public meteorological and agroclimatology data services.
3. ISRIC - World Soil Information, SoilGrids 2.0 - global gridded soil-property information.
4. Kaggle - Crop Yield Prediction Challenge. Dataset licence shown as CC BY 4.0.
5. Kaggle - Regression with a Wild Blueberry Yield Dataset / Wild Blueberry Yield Prediction reference data.
6. Atharva Ingle, Kaggle - Crop Recommendation Dataset. Apache 2.0; contains N, P, K, temperature, humidity, pH and rainfall fields.
7. Kaggle - Tamil Nadu Crop Production / agricultural crop-production analysis reference; official Indian crop statistics are also published via OGD Platform India.
8. Kaggle - Smart Farming Data 2024 (SF24). CC0 Public Domain.
9. Question (e) scaling benchmark and the question-specific demonstrator tables used for reproducible screenshots are generated using NumPy with fixed random seeds and are explicitly labelled generated data in this report.